

PROJET GO · SOUTENANCE

WasmRedis

Un moteur clé-valeur persistant écrit en Go,
et son tableau de bord React

[Parser](#) · [Journal AOF](#) · [Snapshots](#) · [Index typés](#) · [TTL](#) · [API HTTP](#)

Fil rouge : que se passe-t-il, précisément, quand on tape `SET age 42` ?

Le projet en trois phrases

WasmRedis est un **magasin clé-valeur** : il garde des paires clé → valeur en mémoire vive, et les écrit sur le disque pour qu'elles survivent à un redémarrage.

On lui parle en **commandes** — `SET`, `GET`, `DEL`, `PING`, `GET WHERE` — depuis un terminal interactif ou une API HTTP.

Un **tableau de bord React** se branche sur cette API pour piloter des files de tâches et montrer le moteur à l'œuvre.

1 749

lignes de Go
hors tests

7

paquets, sans
dépendance circulaire

2

dépendances externes
btree et flock

28 + 4

tests Go
et tests navigateur

0

commentaire dans
le code source

Le code ne porte aucun commentaire : c'est une règle du projet. Les explications vivent dans le README et dans cette présentation.

Deux moitiés qu'il ne faut pas confondre

Le magasin clé-valeur — persisté

Chaque `SET` et chaque `DEL` finit dans `data/aof.log`, puis dans `data/snapshot.json`.

Arrêtez le programme, relancez-le : les clés sont toujours là.
C'est l'objet des cinq phases de la feuille de route.

Les files de tâches — en mémoire

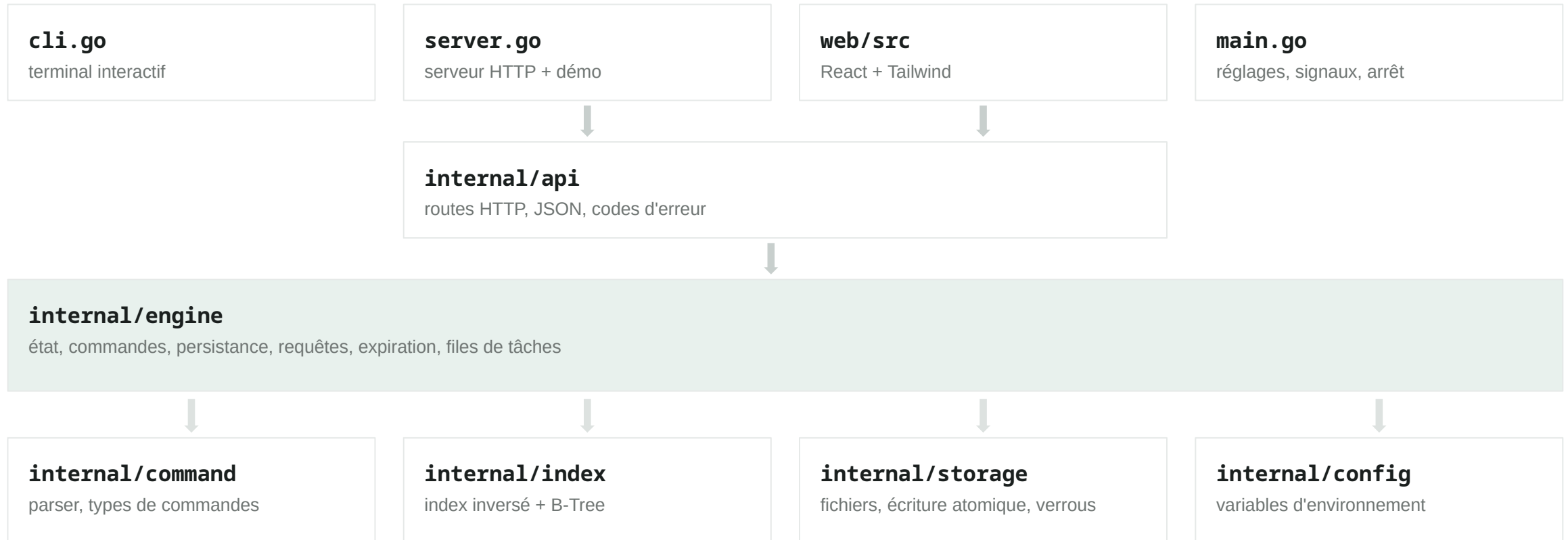
Elles vivent dans une `map` du serveur HTTP, et rien d'autre.
Redémarrez le serveur : les files ont disparu.
Elles servent à démontrer l'API et l'interface, pas à stocker durablement.

Pourquoi cette dissymétrie ?

La persistance demandée portait sur le stockage clé-valeur. Persister aussi les files aurait doublé le travail sans rien démontrer de plus : c'est le même journal, le même snapshot, la même reprise. Le choix est assumé et écrit noir sur blanc dans l'interface elle-même, en bas de l'écran.

« Files en mémoire : elles disparaissent au redémarrage du serveur. » — le texte affiché sous la liste des tâches.

Sept paquets, et des dépendances qui ne remontent jamais



Le moteur connaît la ligne du bas ; la ligne du bas ne connaît pas le moteur. Go refuse les cycles d'import — l'architecture est donc vérifiée par le compilateur.

Tout l'état du moteur tient dans une structure

```
11 type Engine struct {  
12     state      map[string]Entry  
13     mu         sync.Mutex  
14     now         func() time.Time  
15     store       storage.Storage  
16     cfg         config.Config  
17     buffer      []storage.Op  
18     sequence    uint64  
19     closed      bool  
20     running     bool  
21     index       *index.Index  
22 }  
23  
24 type Entry struct {  
25     Key         string    `json:"key"`  
26     Value        string    `json:"value"`  
27     IsNumber     bool      `json:"isNumber"`  
28     ExpiresAt    time.Time `json:"expiresAt"`  
29 }
```

- 1 `state` est une simple `map` : toutes les clés tiennent en mémoire vive.
- 2 Un seul verrou `mu` protège l'ensemble du moteur.
- 3 `now` est une **fonction**, pas un appel à `time.Now` en dur : les tests y injectent une horloge qu'ils contrôlent.
- 4 `buffer` retient les écritures qui n'ont pas encore été versées au journal.
- 5 `sequence` numérote chaque opération — la clé de la reprise après une panne.
- 6 Une valeur, c'est **du texte plus un booléen**. Les comparaisons numériques passent par `float64`.

Un nombre entre guillemets reste une chaîne. Cette décision, prise une seule fois à l'écriture, gouverne ensuite toutes les recherches.

PARTIE 2

Que se passe-t-il quand on fait un SET ?

On suit une seule ligne de texte — SET age 42 — depuis le clavier jusqu'au disque.
Sept étapes, sept fichiers, et une seule d'entre elles touche vraiment le disque.

SET age 42, de bout en bout

ÉTAPE	OÙ	CE QUI SE PASSE
1 Découper la ligne	command/command.go · split	« SET age 42 » devient trois jetons, en retenant lesquels étaient entre guillemets
2 Reconnaître la commande	command/command.go · Parse	Les jetons deviennent une structure Command, ou une erreur
3 Décider du type	command/command.go · numeric	42 est-il un nombre ou une chaîne ? La réponse est figée ici
4 Valider et verrouiller	engine/engine.go · set	Le moteur revérifie tout, puis prend le verrou
5 Écrire en mémoire	engine/engine.go + index	La map et les deux index sont mis à jour ensemble
6 Empiler l'opération	engine/engine.go · record	L'opération numérotée entre dans un tampon ; SET peut retourner OK
7 Écrire sur le disque	storage/storage.go · AppendAOF	Au plus tard une seconde plus tard, le tampon est versé au journal et synchronisé

Les étapes 1 à 6 ne quittent jamais la mémoire. C'est la septième, différée, qui sépare « rapide » de « durable ».

Découper la ligne sans perdre les guillemets

```

96     for _, char := range input {
1  97         if quote != 0 {
98             if char == quote {
99                 quote = 0
100            } else {
101                word.WriteRune(char)
102            }
103            continue
104        }
105        switch char {
2  106        case '\\', '\"':
107            quote = char
108            quoted = true
109            started = true
3  110        case ' ', '\t', '\r', '\n':
111            if started {
4  112                args = append(args, token{text: word.String(), quoted: quoted})
113                word.Reset()
114            }
115        default:
5  122        if quote != 0 {
123            return nil, errors.New("guillemet non fermé")
124        }

```

- 1 Tant qu'un guillemet est ouvert, chaque caractère est pris tel quel : c'est ce qui permet d'écrire `SET nom "Ada Lovelace"`.
- 2 Le guillemet n'est pas conservé dans le texte, mais il laisse une trace : `quoted = true`.
- 3 Hors guillemets, une espace ferme le mot courant. C'est ce qui découpe la ligne en jetons.
- 4 Chaque jeton retient son texte **et** s'il était entre guillemets. Cette seconde information décidera du type à l'étape 3.
- 5 Un guillemet jamais refermé est une erreur franche, pas un mot qui déborde.

La branche `default`, omise ici, se contente d'ajouter le caractère au mot en cours. Les échappements ne sont pas interprétés : c'est un choix de simplicité.

Des jetons à une commande, ou une erreur

```

47     switch cmd.Kind {
48     case KindSet:
1 49         if len(args) != 3 && len(args) != 5 {
50             return Command{}, fmt.Errorf("%w : SET clé valeur [EX secondes]", ErrMissi
                ngArgs)
51         }
2 52         cmd.Key, cmd.Value = args[1].text, args[2].text
3 53         cmd.IsNumber = numeric(args[2])
54         if len(args) == 5 {
4 55             if strings.ToUpper(args[3].text) != "EX" || strings.Trim(args[4].text, "01
                23456789") != "" {
56                 return Command{}, errors.New("expiration attendue : EX secondes (entie
                r positif)")
57             }
5 58             cmd.TTL, err = time.ParseDuration(args[4].text + "s")
59             if err != nil || cmd.TTL <= 0 {
60                 return Command{}, errors.New("la durée EX doit être un entier positif
                représentable")
61             }
62         }

```

- 1 Un SET a soit trois jetons, soit cinq avec l'expiration. Toute autre longueur est refusée avant d'atteindre le moteur.
- 2 La clé et la valeur sont simplement recopiées : le parser ne les transforme pas.
- 3 Le type est décidé ici, une fois pour toutes.
- 4 EX n'accepte que des chiffres : `strings.Trim` retire tous les chiffres et vérifie qu'il ne reste rien — pas de -5, pas de 3.5.
- 5 La durée est reconstruite en secondes, puis revalidée : une durée nulle ou négative est refusée.

Le parser ne touche jamais aux données. Il transforme du texte en une valeur `Command`, ou il renvoie une erreur — et rien n'a bougé.

Une fonction de six lignes décide de tout

```

81 func numeric(value token) bool {
1 82     if value.quoted {
83         return false
84     }
2 85     n, err := strconv.ParseFloat(value.text, 64)
3 86     return err == nil && !math.IsNaN(n) && !math.IsInf(n, 0)
87 }

```

- 1 Un jeton qui était entre guillemets est une chaîne, point final.
- 2 Sinon, on tente une conversion en `float64`.
- 3 NaN et l'infini se convertissent sans erreur mais ne sont pas des nombres utilisables : ils redeviennent des chaînes.

COMMANDE

STOCKÉ COMME

SET age 42

nombre 42

SET age "42"

chaîne « 42 »

SET age 4e3

nombre 4 000

SET age NaN

chaîne « NaN »

SET nom Ada

chaîne « Ada »

Vérifié en vrai : après SET a 4e3, la recherche GET WHERE value >= 3999 trouve la clé — mais GET a rend toujours le texte d'origine, « 4e3 ». Le texte est conservé, le nombre sert à comparer.

Refuser tôt, verrouiller tard

```
63 func (e *Engine) set(key, value string, number bool, ttl time.Duration) error {  
1 64     if ttl < 0 {  
65         return errors.New("le TTL ne peut pas être négatif")  
66     }  
2 67     if number {  
68         n, err := strconv.ParseFloat(value, 64)  
69         if err != nil || math.IsNaN(n) || math.IsInf(n, 0) {  
70             return errors.New("nombre fini requis")  
71         }  
72     }  
3 73     e.mu.Lock()  
4 74     defer e.mu.Unlock()  
75     if e.closed {  
76         return ErrClosed  
77     }  
5 78     if ttl == 0 {  
79         ttl = e.cfg.DefaultTTL  
80     }
```

- 1 Un TTL négatif n'a pas de sens : refus immédiat.
- 2 Si le parser a annoncé un nombre, le moteur le revérifie lui-même — l'API Go est publique et peut être appelée sans passer par le parser.
- 3 **Tout ce qui précède se fait hors du verrou.** On ne bloque les autres appels qu'une fois les validations passées.
- 4 `defer` garantit le déverrouillage, y compris sur les retours d'erreur qui suivent.
- 5 Un SET sans EX hérite du TTL par défaut de la configuration : zéro, donc aucune expiration, sauf réglage contraire.

Valider, puis verrouiller, puis modifier. Une commande destinée à être refusée n'aura jamais fait attendre les autres.

Construire l'entrée, puis publier le changement

```
1 81 entry := Entry{Key: key, Value: value, IsNumber: number}
82 if ttl > 0 {
2 83     entry.ExpiresAt = e.now().Add(ttl)
3 84     if !time.Unix(0, entry.ExpiresAt.UnixNano()).Equal(entry.ExpiresAt) {
85         return errors.New("date d'expiration hors des limites du stockage")
86     }
87 }
4 88 if err := e.index.Set(key, value, number); err != nil {
89     return err
90 }
5 91 e.state[key] = entry
6 92 e.record(command.KindSet, entry)
93 return nil
94 }
```

- 1 L'entrée est construite d'un bloc : clé, valeur, type.
- 2 L'expiration est une **date absolue**, calculée maintenant — pas une durée qui décompte.
- 3 Garde-fou : cette date doit survivre à l'aller-retour en nanosecondes, car c'est sous cette forme qu'elle sera écrite dans le journal.
- 4 L'index est mis à jour **avant** la mémoire. S'il échoue, **e.state** n'a pas bougé : jamais d'index désynchronisé des données.
- 5 L'écriture elle-même : une affectation dans une map. Un **SET** sur une clé existante la remplace, sans cas particulier.
- 6 L'opération est enregistrée pour le journal. À cet instant, elle n'est **pas encore** sur le disque.

Trois lignes suffisent à écrire : index, mémoire, journal. L'ordre entre les trois, lui, n'est pas négociable.

Une écriture, quatre structures tenues à jour

```

49 func (idx *Index) Set(key, text string, number bool) error {
1 50     item, err := scalar(text, number)
51     if err != nil {
52         return err
53     }
2 54     idx.Delete(key)
55     idx.entries[key] = item
3 56     idx.keys.ReplaceOrInsert(key)
57     if idx.inverted[item] == nil {
58         idx.inverted[item] = map[string]struct{}{}
4 59         idx.values.ReplaceOrInsert(item)
60     }
5 61     idx.inverted[item][key] = struct{}{}
62     return nil
63 }

```

- 1 `scalar` normalise : un nombre est converti puis reformaté. `4e3` et `4000` deviennent la même entrée d'index.
- 2 Remplacer, c'est d'abord supprimer. Sans cette ligne, l'ancienne valeur garderait la clé dans son ensemble et `GET WHERE` mentirait.
- 3 L'arbre des clés, pour les recherches `GET WHERE key > ...`.
- 4 Une valeur encore inconnue crée son ensemble et entre dans l'arbre des valeurs.
- 5 L'index inversé : valeur → ensemble de clés. `equals` ne sera plus qu'une lecture de map.

Quatre structures pour un seul fait — « la clé K vaut V ». C'est le prix d'une recherche rapide, payé à chaque écriture.

L'opération entre dans la file d'attente du disque

```
137 func (e *Engine) record(kind string, entry Entry) {  
1 138     if e.store == nil {  
139         return  
140     }  
2 141     e.sequence++  
3 142     op := storage.Op{Sequence: e.sequence, Kind: kind, Key: entry.Key, Value: entry.Value, IsNumber: entry.IsNumber}  
4 143     if !entry.ExpiresAt.IsZero() {  
144         op.ExpiresAt = entry.ExpiresAt.UnixNano()  
145     }  
5 146     e.buffer = append(e.buffer, op)  
147 }
```

- 1 En mode `-memory`, il n'y a pas de stockage : la fonction ne fait rien et tout le reste du moteur est identique.
- 2 Chaque opération reçoit un numéro qui ne recule jamais.
- 3 L'opération journalisée est autosuffisante : elle contient tout ce qu'il faut pour être rejouée seule.
- 4 L'expiration devient un entier — des nanosecondes depuis 1970 — parce que c'est ce qui s'écrit proprement en JSON.
- 5 Elle est simplement ajoutée au tampon, et `SET` peut retourner `OK`. **L'appelant n'a pas attendu le disque.**

C'est la raison pour laquelle un arrêt brutal peut perdre les toutes dernières écritures. Le compromis est assumé, et documenté.

Vider le tampon, ou ne rien perdre en essayant

```
1 14 func (e *Engine) Flush() error {  
15     e.mu.Lock()  
16     defer e.mu.Unlock()  
17     return e.flush()  
18 }  
19  
20 func (e *Engine) flush() error {  
2 21     if e.store == nil || len(e.buffer) == 0 {  
22         return nil  
23     }  
3 24     if err := e.store.AppendAOF(e.buffer); err != nil {  
4 25         return err  
26     }  
5 27     e.buffer = nil  
28     return nil  
29 }
```

- 1 La version publique prend le verrou, la version privée suppose qu'il est déjà pris. Ce doublet se retrouve partout dans le moteur.
- 2 Sans stockage, ou sans rien à écrire, il n'y a rien à faire.
- 3 Une seule écriture pour tout le tampon, pas une écriture par opération.
- 4 **En cas d'échec, on retourne sans vider le tampon.** Les opérations restent en mémoire et seront réessayées au flush suivant.
- 5 Le tampon n'est remis à zéro qu'après un succès confirmé.

Une tâche de fond appelle Flush toutes les secondes. Close en fait un dernier à l'arrêt, et POST /api/flush permet de le déclencher à la demande.

Le contrat avec le système de fichiers

```

22     var data bytes.Buffer
23     encoder := json.NewEncoder(&data)
1 24     for _, op := range ops {
25         if err := encoder.Encode(op); err != nil {
26             return err
27         }
28     }
    :
2 32     file, err := os.OpenFile(s.aofPath, os.O_CREATE|os.O_RDWR|os.O_APPEND, 0600)
33     if err != nil {
34         return err
35     }
36     defer file.Close()
3 37     info, err := file.Stat()
38     if err != nil {
39         return err
40     }
4 41     if _, err = file.Write(data.Bytes()); err == nil {
42         err = file.Sync()
43     }
44     if err != nil {
5 45         return errors.Join(err, file.Truncate(info.Size()), file.Sync())
46     }
6 47     return syncDir(s.aofPath)

```

- 1 Tout le JSON est fabriqué en mémoire d'abord, une ligne par opération. Si l'encodage échoue, le fichier n'a pas été touché.
- 2 Ouverture en **ajout** : on n'écrase jamais ce qui est déjà écrit.
- 3 On relève la taille du fichier **avant** d'écrire. C'est le point de retour.
- 4 **Write** puis **Sync** : c'est **Sync** qui force le système à poser réellement les octets sur le disque.
- 5 Si quelque chose échoue, le fichier est retronqué à sa taille d'avant : le journal ne conserve jamais une ligne à moitié écrite.
- 6 On synchronise aussi le **dossier** — sinon la création du fichier elle-même pourrait ne pas survivre à une coupure de courant.

C'est ici, et seulement ici, que « la donnée est enregistrée » devient vrai.

Une vraie session, copiée telle quelle

```
$ go run . -cli
restauration : 0 clé en 0s

> SET nom "Ada Lovelace"
OK
> SET age 42
OK
> SET etiquette "42"
OK
> SET code secret EX 60
OK
> GET age
42
> GET WHERE value >= 21
[{"key":"age","value":"42","isNumber":true,"expiresAt":"0001-01-01T00:00:00Z"}]
> DEL etiquette
OK
> EXIT
moteur fermé : 5 opérations écrites au dernier flush
```

- 1 GET et PING ne produisent aucune opération : seules les écritures entrent dans le journal.
- 2 GET WHERE value >= 21 rend age et ignore etiquette, dont la valeur « 42 » est une chaîne. Même texte, type différent, résultat différent.
- 3 À la fermeture, le moteur annonce cinq opérations écrites : quatre SET et un DEL.
- 4 Les cinq ont attendu dans le tampon ; aucune n'a fait patienter l'utilisateur.

Ce que le journal a retenu — data/aof.log

```
{ "sequence":1, "kind":"SET", "key":"nom", "value":"Ada Lovelace", "isNumber":false, "expiresAt":0 }
{ "sequence":2, "kind":"SET", "key":"age", "value":"42", "isNumber":true, "expiresAt":0 }
{ "sequence":3, "kind":"SET", "key":"etiquette", "value":"42", "isNumber":false, "expiresAt":0 }
{ "sequence":4, "kind":"SET", "key":"code", "value":"secret", "isNumber":false, "expiresAt":1789545818655556325 }
{ "sequence":5, "kind":"DELETE", "key":"etiquette", "value":""," "isNumber":false, "expiresAt":0 }
```

- 1 Une ligne = une opération, au format **JSON Lines**. Le retour à la ligne final fait partie du contrat.
- 2 Les numéros de séquence se suivent. Un trou signalerait un journal corrompu, et le démarrage échouerait.
- 3 `isNumber` sépare `age` de `etiquette` : la même valeur « 42 », écrite deux fois, avec deux types.
- 4 L'expiration est une date absolue en nanosecondes depuis 1970. Zéro signifie « jamais ».
- 5 La suppression est une ligne **en plus**, pas une ligne en moins. Le journal ne se réécrit pas : il s'allonge, jusqu'au prochain snapshot.

Ce fichier se lit avec `cat`. C'était un objectif : pouvoir vérifier à l'œil ce que le moteur a promis.

PARTIE 3

Lire, chercher, oublier

GET, DELETE, GET WHERE et l'expiration : le reste du vocabulaire du moteur.
C'est là que le choix de typer les valeurs se paie — et se justifie.

GET, et l'oubli paresseux

```
96 func (e *Engine) Get(key string) (string, error) {
97     entry, err := e.getEntry(key)
98     return entry.Value, err
99 }
100
101 func (e *Engine) getEntry(key string) (Entry, error) {
102     e.mu.Lock()
103     defer e.mu.Unlock()
104     entry, exists := e.state[key]
105     if !exists {
106         return Entry{}, ErrNotFound
107     }
108     if expired(entry, e.now()) {
109         if !e.closed {
110             e.remove(key)
111         }
112         return Entry{}, ErrNotFound
113     }
114     return entry, nil
115 }
116
117 func expired(entry Entry, now time.Time) bool {
118     return !entry.ExpiresAt.IsZero() && !now.Before(entry.ExpiresAt)
119 }
```

- 1 La lecture, c'est un accès à la map. Rien de plus.
- 2 Une clé peut être présente **et** périmée : le test d'expiration se fait au moment de la lecture.
- 3 Sauf si le moteur est fermé — on ne journalise plus rien après `Close`.
- 4 Sinon la clé périmée est supprimée au passage : la lecture fait le ménage, index et journal compris.
- 5 Une date d'expiration nulle signifie « jamais ». À l'instant exact de l'expiration, la clé est déjà considérée comme périmée.

Conséquence : le moteur ne rend jamais une clé périmée, même si le balayage de fond n'est pas encore passé.

DELETE, ou une ligne de plus dans le journal

```
121 func (e *Engine) Delete(key string) error {  
122     e.mu.Lock()  
123     defer e.mu.Unlock()  
1 124     if e.closed {  
125         return ErrClosed  
126     }  
2 127     e.remove(key)  
128     return nil  
129 }  
130  
131 func (e *Engine) remove(key string) {  
3 132     e.index.Delete(key)  
4 133     delete(e.state, key)  
5 134     e.record(command.KindDelete, Entry{Key: key})  
135 }
```

- 1 Après `Close`, on refuse l'écriture plutôt que de la perdre en silence.
- 2 `Delete` et l'expiration passent par le même chemin : `remove`.
- 3 L'index est nettoyé en premier, comme à l'écriture.
- 4 `delete` sur une clé absente ne fait rien. Supprimer ce qui n'existe pas n'est pas une erreur.
- 5 La suppression est journalisée comme une opération à part entière, avec son propre numéro de séquence.

Le journal ne rétrécit jamais lors d'un DEL : il s'allonge d'une ligne qui dit « oublie celle-ci ». Seul le snapshot fait maigrir le fichier.

Quatre structures pour ne pas tout parcourir

Le problème : trouver toutes les clés dont la valeur dépasse 21, sans lire toutes les clés.

```
5  type value struct {  
1  6      text    string  
7      number bool  
8      numeric float64  
9  }  
10  
11 type Index struct {  
2  12     keys      *btree.BTreeG[string]  
3  13     values    *btree.BTreeG[value]  
4  14     entries   map[string]value  
5  15     inverted  map[value]map[string]struct{}  
16 }
```

- 1 Une `value` retient son texte **et** son équivalent `float64`, plus le drapeau qui dit lequel fait foi.
- 2 `keys` : un B-Tree ordonné des clés, pour les plages sur `key`.
- 3 `values` : un B-Tree ordonné des valeurs, pour les plages sur `value`.
- 4 `entries` : clé → valeur. Sert à retrouver l'ancienne valeur lors d'un remplacement.
- 5 `inverted` : valeur → ensemble des clés. C'est l'index inversé, et `equals` y lit directement.

Le B-Tree vient de github.com/google/btree. Écrire le nôtre aurait triplé la taille du paquet sans rien apprendre de plus sur le sujet du projet.

equals et contains : les deux extrêmes

```

95     keys := []string{}
1  96     if operator == "contains" {
97         for key, item := range idx.entries {
2  98             if field == "key" && strings.Contains(key, operand) || field == "v
           aalue" && !item.number && strings.Contains(item.text, operand) {
99                 keys = append(keys, key)
100             }
101         }
           :
134         case "equals":
3  135         for key := range idx.inverted[pivot] {
136             keys = append(keys, key)
137         }
           :
4  144     sort.Strings(keys)
145     return keys, nil

```

- 1 `contains` n'a aucun index : il parcourt toutes les entrées, une par une.
- 2 Sur une valeur, `contains` ignore les nombres — chercher « 4 » dans 42 n'aurait pas de sens ici.
- 3 `equals`, à l'inverse, est une simple lecture de l'index inversé. Son coût dépend du nombre de résultats, pas du nombre de clés.
- 4 Tous les chemins finissent triés par clé : le résultat est stable, alors que l'ordre de parcours d'une map en Go ne l'est pas.

Deux extrêmes dans la même fonction. Le coût de `contains` est assumé et écrit dans le README : indexer les sous-chaînes aurait coûté bien plus cher à chaque écriture.

Les plages : descendre dans le B-Tree

```

1 121      visit := func(item value) bool {
2 122          if item.number != pivot.number {
123              return false
124          }
3 125          if (operator == ">" || operator == "<") && item == pivot {
126              return true
127          }
4 128          for key := range idx.inverted[item] {
129              keys = append(keys, key)
130          }
131          return true
132      }

:

138      case ">", ">=":
5 139          idx.values.AscendGreaterOrEqual(pivot, visit)
140      case "<", "<=":
6 141          idx.values.DescendLessOrEqual(pivot, visit)
142      }

```

- 1 `visit` est appelé par l'arbre pour chaque valeur rencontrée, dans l'ordre.
- 2 **Dès qu'on change de type, on s'arrête.** Renvoyer `false` interrompt le parcours.
- 3 Sur `>` et `<` stricts, la borne elle-même est visitée mais pas retenue.
- 4 Pour chaque valeur retenue, on récupère d'un coup toutes ses clés dans l'index inversé.
- 5 L'arbre se positionne sur la borne et remonte en ordre croissant, sans jamais regarder ce qui est en dessous.
- 6 Et symétriquement pour les bornes inférieures.

Sans l'arbre, il faudrait comparer les 100 000 valeurs pour en trouver trois. Avec lui, on entre à la bonne place et on s'arrête dès que la condition tombe.

Nombres et chaînes ne se mélangent pas

```
23 func less(a, b value) bool {  
1 24     if a.number != b.number {  
2 25         return !a.number  
26     }  
27     if a.number {  
3 28         return a.numeric < b.numeric  
29     }  
4 30     return a.text < b.text  
31 }
```

```
> SET age 42  
> SET etiquette "42"  
> GET WHERE value >= 21  
[{"key":"age","value":"42","isNumber":true,"expiresAt":"0001-01-01T00:00:00Z"}]  
une seule clé : « 42 » entre guillemets est une chaîne
```

- 1 Deux valeurs de types différents ne se comparent pas sur leur contenu.
- 2 Elles sont rangées en deux blocs : toutes les chaînes d'un côté, tous les nombres de l'autre.
- 3 Entre nombres, on compare des `float64`.
- 4 Entre chaînes, l'ordre lexicographique.

C'est la question que pose toujours un jury : « et si je stocke 42 et "42" ? » Réponse : deux valeurs distinctes, dans deux moitiés distinctes de l'arbre, et une recherche numérique ne verra jamais la seconde.

TTL : deux chemins mènent à l'oubli

```
9 func (e *Engine) SweepExpired() int {
10     e.mu.Lock()
11     defer e.mu.Unlock()
12     if e.closed {
13         return 0
14     }
15     removed := 0
16     now := e.now()
17     for key, entry := range e.state {
18         if expired(entry, now) {
19             e.remove(key)
20             removed++
21         }
22     }
23     return removed
24 }
```

- 1 Après fermeture, le balayage ne fait plus rien : personne ne pourrait plus écrire son résultat.
- 2 L'heure est relevée **une seule fois** pour tout le balayage — deux clés qui expirent en même temps disparaissent ensemble.
- 3 Le balayage est complet : toutes les clés, à chaque passage. Simple, et suffisant à cette échelle.
- 4 Le même `remove` que `DEL` : index nettoyé et suppression journalisée.

La lecture garantit la justesse — elle ne rend jamais une clé périmée. Le balayage, lui, libère la mémoire des clés que personne ne lit plus. Toutes les dix secondes par défaut.

PARTIE 4

Survivre à un redémarrage

Un journal qui s'allonge, un snapshot qui le remplace, et une reprise qui recolle les deux.
Toute la difficulté tient dans un mot : que se passe-t-il si le courant coupe ici ?

Le snapshot, puis le journal — jamais l'inverse

```
31 func (e *Engine) Snapshot() error {
32     e.mu.Lock()
33     defer e.mu.Unlock()
34     if e.store == nil {
35         return nil
36     }
37     if err := e.flush(); err != nil {
38         return err
39     }
40     snapshot := storage.Snapshot{Sequence: e.sequence, Entries: map[string]storage.Op{}}
41     for key, entry := range e.state {
42         if expired(entry, e.now()) {
43             continue
44         }
45         op := storage.Op{Kind: command.KindSet, Key: key, Value: entry.Value, IsNumber:
46             entry.IsNumber}
47         :
48         snapshot.Entries[key] = op
49     }
50     if err := e.store.WriteSnapshot(snapshot); err != nil {
51         return err
52     }
53     return e.store.ClearAOF()
54 }
55 }
```

- 1 On commence par vider le tampon : le snapshot doit refléter tout ce qui a déjà été accepté.
- 2 Le snapshot retient le numéro de séquence atteint. C'est lui qui dira, au redémarrage, quelles lignes du journal sont déjà couvertes.
- 3 Les clés périmées ne sont pas recopiées : le snapshot fait aussi le ménage.
- 4 Le fichier est écrit — de façon atomique, on y vient.
- 5 **Et seulement ensuite** le journal est vidé.

Si le programme meurt entre l'étape 4 et l'étape 5, le journal contient des lignes déjà couvertes par le snapshot. Les numéros de séquence les rendent inoffensives.

Fichier temporaire, synchronisation, renommage

```
125 func writeAtomic(path string, data []byte) error {
126     if err := os.MkdirAll(filepath.Dir(path), 0700); err != nil {
127         return err
128     }
129     1 file, err := os.CreateTemp(filepath.Dir(path), ".wasmredis-*")
130     if err != nil {
131         return err
132     }
133     2 defer os.Remove(file.Name())
134     3 if _, err = file.Write(data); err == nil {
135         err = file.Sync()
136     }
137     4 err = errors.Join(err, file.Close())
138     if err != nil {
139         return err
140     }
141     5 if err := os.Rename(file.Name(), path); err != nil {
142         return err
143     }
144     6 return syncDir(path)
145 }
```

- 1 On écrit dans un fichier temporaire placé **à côté** de la cible — même dossier, donc même système de fichiers.
- 2 Si quoi que ce soit échoue ensuite, le temporaire est effacé et le fichier visé n'aura jamais été touché.
- 3 Écriture puis **Sync** : les octets sont réellement sur le disque avant qu'on aille plus loin.
- 4 **errors.Join** conserve les deux erreurs possibles, celle de l'écriture et celle de la fermeture.
- 5 **Rename** sur un même système de fichiers est **atomique** : à tout instant, la cible est soit l'ancien fichier entier, soit le nouveau entier.
- 6 On synchronise aussi le dossier, pour que le renommage lui-même survive à une coupure.

Jamais de fichier à moitié écrit. C'est ce qui autorise à couper le courant pendant un snapshot sans perdre le précédent.

Le redémarrage : snapshot d'abord, journal ensuite

```
83     sequence := snapshot.Sequence
1 84     for _, op := range ops {
85         if op.Sequence == 0 {
86             return errors.New("séquence du journal invalide")
87         }
2 88         if op.Sequence <= snapshot.Sequence {
89             continue
90         }
91         if op.Sequence <= sequence {
92             continue
93         }
3 94         if op.Sequence != sequence+1 {
95             return errors.New("séquence du journal interrompue")
96         }
4 97         if err := applyOp(state, op); err != nil {
98             return err
99         }
5 100         sequence = op.Sequence
101     }
```

- 1 Le snapshot est déjà chargé. On rejoue maintenant le journal par-dessus.
- 2 Les lignes déjà couvertes par le snapshot sont ignorées : c'est le filet de sécurité d'une compaction interrompue.
- 3 Un trou dans la numérotation arrête le démarrage. Mieux vaut refuser d'ouvrir que servir des données incomplètes.
- 4 `applyOp` refait l'opération : un `SET` réécrit, un `DELETE` efface.
- 5 Et le compteur avance, pour que la prochaine écriture reprenne la suite.

Après le rejeu, les clés périmées sont éliminées et les index reconstruits de zéro. Ils ne sont jamais persistés : les recalculer est plus simple, et plus sûr, que les stocker.

Une écriture interrompue n'est pas une corruption

```
1 60     end := bytes.LastIndexByte(data, '\n') + 1
61     ops := []Op{}
2 62     for i, line := range bytes.Split(data[:end], []byte{'\n'}) {
63         if len(line) == 0 {
64             continue
65         }
66         var op Op
3 67         if err := json.Unmarshal(line, &op); err != nil {
68             return nil, fmt.Errorf("journal ligne %d : %w", i+1, err)
69         }
70         ops = append(ops, op)
71     }
```

- 1 On ne lit que jusqu'au **dernier retour à la ligne**. Ce qui suit est une écriture coupée en vol : elle est ignorée, puis retirée du fichier.
- 2 Chaque ligne complète est une opération.
- 3 Une ligne complète mais illisible, elle, est une vraie corruption. Le démarrage échoue en nommant la ligne fautive.

La différence est nette : une ligne tronquée est un accident normal de la coupure de courant ; un octet modifié au milieu d'une ligne complète ne l'est pas.

Et si ça coupe ici ?

CE QUI ARRIVE

Coupure pendant un flush

Coupure pendant un snapshot

Coupure entre le snapshot et le vidage du journal

Octet corrompu dans une ligne complète

Arrêt brutal (kill -9, coupure)

Deux processus sur les mêmes fichiers

CE QUI SE PASSE ALORS

La dernière ligne, incomplète, est retirée au redémarrage. Les précédentes sont intactes.

Le fichier temporaire est abandonné. L'ancien snapshot et le journal restent valides.

Le journal contient des lignes déjà couvertes : les numéros de séquence les font ignorer.

Le démarrage échoue et nomme la ligne. Pas de démarrage silencieux sur des données fausses.

Les opérations encore dans le tampon sont perdues. C'est la limite assumée du tampon d'une seconde.

Le second refuse de démarrer : LockFiles pose un verrou flock sur le journal et le snapshot.

Un arrêt propre — Ctrl-C, SIGTERM — passe par `Engine.Close()`, qui fait un dernier flush et journalise le nombre d'opérations écrites. Rien n'est perdu.

PARTIE 5

Autour du moteur

Le cycle de vie du programme, l'API HTTP, le tableau de bord et les vérifications.
Ce qui transforme un paquet Go en quelque chose qu'on peut lancer et montrer.

Démarrer, écouter les signaux, s'arrêter proprement

```

81  stopping := make(chan os.Signal, 1)
1  82  signal.Notify(stopping, os.Interrupt, syscall.SIGTERM)
83  defer signal.Stop(stopping)
2  84  ctx, cancel := context.WithCancel(context.Background())
85  defer cancel()
86  go func() {
87      select {
3  88      case received := <-stopping:
89          log.Printf("signal %s reçu : arrêt propre, écriture du tampon avant la sor
0          tie", received)
90          cancel()
91      case <-ctx.Done():
92      }
93  }()
94  background := make(chan error, 1)
4  95  go func() { background <- eng.StartBackground(ctx); cancel() }()
96  if *cli {
97      result = runCLI(ctx, eng, *memory)
98  } else {
99      result = runServer(ctx, *addr, *demo, *demoTasks, eng)
100 }
5 101 cancel()
6 102 return errors.Join(result, <-background)

```

- 1 Le programme s'abonne à `Ctrl-C` et à `SIGTERM`.
- 2 Un `context` annulable sert de signal d'arrêt unique pour tout le programme.
- 3 Une goroutine attend le signal et annule le contexte. Un message explique ce qui va se passer.
- 4 Les tâches de fond — flush, snapshot, expiration — tournent dans leur propre goroutine et remontent leurs erreurs par un canal.
- 5 À la sortie, on annule d'abord, pour que les tâches de fond s'arrêtent.
- 6 Puis on attend leur erreur éventuelle et on la joint au résultat. Les `defer` ferment ensuite le moteur, puis libèrent les verrous.

L'ordre de l'arrêt est le contraire de celui du démarrage : tâches de fond, puis `Engine.Close()` qui écrit le tampon, puis les verrous de fichiers.

Un seul verrou, et ce qu'il coûte

Ce qu'on y gagne

Aucune lecture ne peut voir un état à moitié écrit.

Les index et les données sont toujours d'accord entre eux.

Le raisonnement tient en une phrase : une opération à la fois.

Ce qu'on y paie

Deux lectures ne se font pas en parallèle.

Un balayage d'expiration bloque les écritures le temps du passage.

La montée en charge s'arrête au débit d'un seul cœur.

Ce qui est vérifié

`go test -race ./...` couvre les accès concurrents et les tâches de fond.

Les tests injectent une horloge et un stockage qui simule les pannes disque.

Le tampon, l'état et les index sont observés depuis l'intérieur du paquet.

Ce qu'on ferait ensuite

Un `sync.RWMutex` laisserait les lectures passer ensemble.

Un découpage de la map en tranches réduirait la contention.

Aucun des deux n'était nécessaire au périmètre demandé.

Une route, et le même moteur que le terminal

```
1 84 mux.HandleFunc("POST /api/command", func(w http.ResponseWriter, r *http.Request) {
85     var body commandRequest
2 86     if !read(w, r, &body) {
87         return
88     }
3 89     result := kv.ExecuteString(body.Command)
90     if result.Err != nil {
4 91         write(w, http.StatusBadRequest, map[string]string{"error": result.Err.Erro
           r()})
92         return
93     }
94     write(w, http.StatusOK, response(result))
95 })
```

- 1 Depuis Go 1.22, la méthode fait partie du motif de route : pas de routeur externe à installer.
- 2 `read` refuse les corps de plus de 16 Kio, les champs inconnus et le JSON en trop après l'objet.
- 3 Le serveur ne réimplémente rien : il passe la chaîne au même `ExecuteString` que le terminal interactif.
- 4 Une commande invalide est une faute du client : 400, pas 500. Un échec d'écriture disque, lui, donne bien 500.

Routes : POST /api/command, POST /api/batch, POST /api/flush, POST /api/snapshot, plus les six routes des files de tâches. Le reste sert le tableau de bord compilé.

Le batch n'est pas une transaction

Envoyer cent commandes en une requête, sans prétendre qu'elles forment un tout indivisible.

```
5 func (e *Engine) ExecuteBatch(commands []command.Command) []Result {  
1 6     results := make([]Result, len(commands))  
7     for i, cmd := range commands {  
2 8         results[i] = e.Execute(cmd)  
9     }  
10    return results  
11 }  
12  
3 13 func (e *Engine) ExecuteBatchStrings(lines []string) []Result {  
14     results := make([]Result, len(lines))  
15     for i, line := range lines {  
16         results[i] = e.ExecuteString(line)  
17     }  
18     return results  
19 }
```

- 1 Le tableau des résultats a exactement la taille du tableau des commandes.
- 2 **Une erreur n'interrompt pas les suivantes :** chaque commande a son propre résultat, à sa propre place.
- 3 La variante qui prend des chaînes fait le même travail, en passant d'abord par le parser.

Ce n'est pas une transaction : un autre client peut écrire entre deux commandes du lot, et rien n'est annulé en cas d'erreur. La limite est de 1 000 commandes et 16 Kio par requête.

Des files de tâches, pour montrer le moteur à l'œuvre

Files FIFO, quatre statuts, transitions manuelles — et rien qui exécute le contenu des tâches.

WasmRedis

emails1 000

exports1 000

images1 000

Nouvelle file

ex. notifications

+

emails

970 en attente 5 en cours 20 terminées 5 en erreur

Démarrer la suivante

Nouvelle tâche...

+ Ajouter une tâche

StatutTous les statuts

1 000 tâches

ID	TÂCHE	STATUT	ACTIONS
#1	Tâche emails 0001	16/09/2026 10:07:27	En erreurRéessayer
#2	Tâche emails 0002	16/09/2026 10:07:27	Terminées
#3	Tâche emails 0003	16/09/2026 10:07:27	Terminées
#4	Tâche emails 0004	16/09/2026 10:07:27	Terminées
#5	Tâche emails 0005	16/09/2026 10:07:27	Terminées
#6	Tâche emails 0006	16/09/2026 10:07:27	En erreurRéessayer
#7	Tâche emails 0007	16/09/2026 10:07:27	Terminées
#8	Tâche emails 0008	16/09/2026 10:07:27	Terminées

Files en mémoire : elles disparaissent au redémarrage du serveur.

- Une file, c'est une liste de tâches **et quatre compteurs** tenus à jour à chaque transition : `List` ne parcourt plus jamais les tâches.
- Les transitions autorisées tiennent en une condition : en attente → en cours → terminée ou en erreur, et en erreur → en attente.
- Le filtrage passe par la liste déroulante ; les compteurs sous le titre sont informatifs.
- L'interface interroge l'API toutes les deux secondes, en annulant proprement la requête précédente.
- `-demo-tasks` `N` fixe le nombre de tâches par file — trois files de mille par défaut.

Rappel : ces files vivent en mémoire et disparaissent à l'arrêt du serveur. Le magasin clé-valeur, lui, est sur le disque.

Afficher mille lignes sans en construire mille

```

13 const ROW_HEIGHT = 56;
14 const PAGE_SIZE = 100;
15 const OVERSCAN = 4;
1 16 const MAX_SPACE = 30000000;
    :
23   const scaled = total * ROW_HEIGHT > MAX_SPACE;
2 24   const space = scaled ? MAX_SPACE : total * ROW_HEIGHT;
    :
3 30   const anchor = scaled
31       ? Math.round(ratio * Math.max(0, total - visible))
32       : Math.floor(scrollTop / ROW_HEIGHT);
4 33   const first = Math.max(0, anchor - OVERSCAN);
34   const last = Math.min(total, anchor + visible + OVERSCAN);
    :
5 38   const offset = Math.floor(first / PAGE_SIZE) * PAGE_SIZE;
39   const path = `${base}/tasks?offset=${offset}&limit=200&status=${filter}`
    ;

```

- 1 Au-delà d'environ 33 millions de pixels, un navigateur cesse de gérer la hauteur d'un élément. On plafonne en dessous.
- 2 Sous ce plafond, la hauteur totale est exacte : une ligne vaut 56 pixels, et le placement est au pixel près.
- 3 Au-dessus, la ligne du haut se déduit d'un simple rapport. Le défilement devient plus rapide qu'une ligne par ligne : c'est le compromis assumé.
- 4 On ne construit que la fenêtre visible, plus quatre lignes de marge de chaque côté.
- 5 Et on ne demande au serveur que la page de cent qui contient cette fenêtre.

ID	TÂCHE	STATUT	ACTIONS
#930	Tâche emails 0930 16/09/2026 10:07:27	● En attente	
#931	Tâche emails 0931 16/09/2026 10:07:27	● En attente	

Après avoir fait défiler jusqu'au fond de la file, la liste montre la 930^e tâche. Le document, lui, ne contient qu'une trentaine de lignes.

Cinq commandes, et ce qu'elles couvrent

COMMANDE

CE QU'ELLE VÉRIFIE

go test -race ./...

28 fonctions de test sur 6 paquets, accès concurrents compris

go vet ./...

Les fautes courantes que le compilateur laisse passer

go build ./...

La compilation de tous les paquets

npm run build --prefix web

La construction du tableau de bord

npm run test:e2e --prefix web

4 scénarios navigateur, sur un serveur isolé

LES QUATRE SCÉNARIOS NAVIGATEUR

- Créer une file et suivre le cycle complet d'une tâche
- Virtualiser mille tâches et charger la fin de la file
- Afficher une erreur réseau, puis récupérer automatiquement
- Vérifier que l'interface reste utilisable sur mobile

CE QUE LES TESTS GO SAVENT FAIRE

- Injecter une horloge, pour tester le TTL sans attendre
- Simuler une panne disque et vérifier que le tampon est conservé
- Tronquer un journal pour rejouer une coupure

Les cinq commandes passent au vert sur ce dépôt. Les tests Go restent dans le paquet qu'ils testent : six fichiers ont besoin de champs non exportés.

Ce que ce projet ne fait pas

Limites assumées du moteur

Un seul verrou global : pas de lectures parallèles.

Le balayage d'expiration est complet à chaque passage.

`contains` parcourt toutes les entrées, sans index.

Les recherches ne sont pas paginées : elles rendent tout.

Un arrêt brutal perd le tampon de la dernière seconde.

Hors périmètre, et c'était prévu

Pas de protocole réseau Redis, pas d'authentification.

Les files de tâches ne sont pas persistées.

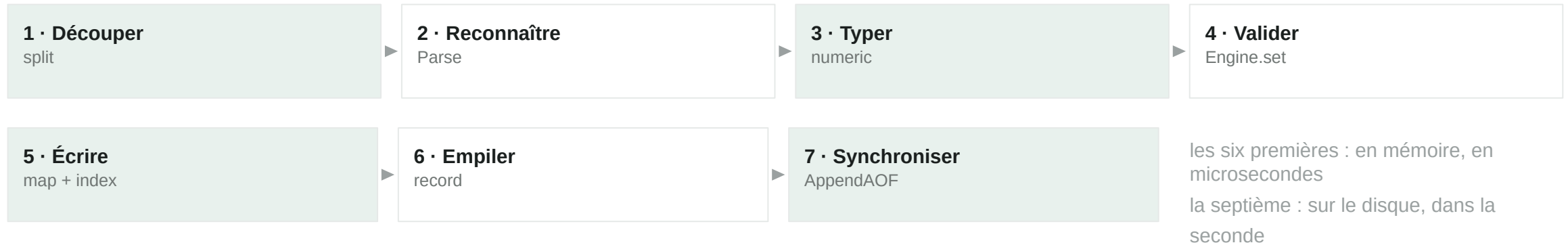
Aucun worker n'exécute le contenu des tâches.

WebAssembly, OPFS, worker navigateur et SDK TypeScript appartiennent aux lots ultérieurs.

Pourquoi les nommer plutôt que les taire ?

Parce que chacune de ces limites est le revers d'un choix explicite, et qu'aucune n'est un oubli. Le verrou unique achète la simplicité du raisonnement ; le tampon d'une seconde achète la vitesse d'écriture ; l'absence d'index sur `contains` évite d'alourdir chaque `SET` pour une fonctionnalité secondaire. Un jury peut discuter le compromis — c'est bien qu'il y en ait un.

SET age 42, en une image



Une décision prise une fois

Nombre ou chaîne : le type est fixé à l'écriture, et toutes les recherches en découlent.

Un ordre qui ne change jamais

Valider, verrouiller, écrire en mémoire, journaliser, synchroniser. Chaque inversion serait un bug.

Une panne prévue à chaque étape

Ligne tronquée, snapshot interrompu, disque plein : chaque cas a une réponse écrite et testée.

Merci. Le code, le README et cette présentation disent la même chose — c'était le but.